



Rayat Shikshan Sanstha's

R.B. Narayanrao Borawake College, Shrirampur

(Autonomous)

National Education Policy

(Affiliated to Savitribai Phule Pune University)

**Two Years Degree Program in Computer Science
(Faculty of Science and Technology)**

Syllabus under Autonomy and National Education Policy

M. Sc. (Computer Science) Part- I

Practical Assignment Lab Book

**Choice Based Credit System [CBCS] Syllabus for National
Education Policy to be implemented from Academic Year**

2025-2026

Advance Operating System (CS-MJ-511T)

Assignment Completion Sheet

Name of the Student:

Roll No:

Sr. No.	Assignment Title	Marks Obtained	Signature of Instructor
1	Print "HELLO WORLD" without using printf().		
2	Accept file from user or as command line arguments and print its inode number.		
3	Write c program to find properties such as inode number, number of hard link, file permissions, file size, file access and modification time and so on of a given file using stat() function.		
4	Print the type and size of a given file.		
5	Write a c program to find whether a given file is present in current directory or not.		
6	Write a c program that accept a string as an argument and return all the files that begins with that name in the current directory .for e.g. ./a.out sss will return all file names that begins with sss.		
7	Read the current directory and display name of files ,no of files in current directory.		
8	Write c program to print all environment variables		
9	Atexit() system call execution for exit_handler().		
10	Fork(), vfork() system call execution		
11	Write a C Program that demonstrates redirection of standard output to a file.		
12	Write a C program that illustrates suspending and resuming processes using signals.		
13	Demonstrate the use of atexit() function.		
14	Message 1= "Hello SPPU".		

15	Message 3 = “Linux is Funny”		
----	------------------------------	--	--

Total Marks :

Converted into 10 Marks :

Date:

Signature of In Charge

Head

Internal Examiner

External Examiner

Assignment No.1

Q.1) Print “HELLO WORLD” without using printf().

```
#include<stdio.h>
int main()
{
    puts(“HELLO WORLD”);
    return(0);
}
```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment No .2

Q.2) Accept file from user or as command line arguments and print its inode number.

Method 1: Accept filepath from user:

```
#include <stdio.h>
#include <sys/stat.h>

int main()
{
    char filePath[256];
    struct stat fileStat;

    printf("Enter file path: ");
    scanf("%s",& filePath);

    if (stat(filePath, &fileStat) == 0) {
        printf("Inode number: %lu\n", fileStat.st_ino);
    } else {
        perror("stat");
    }

    return 0;
}
```

Method 2: Taking file path as command-line argument

```
#include <stdio.h>
#include <sys/stat.h>

int main(int argc, char *argv[])
{
    struct stat fileStat;

    if (stat(argv[1], &fileStat) == 0)
```

```

    {
        printf("Inode number: %lu\n", fileStat.st_ino);
    }
else
{
    perror("stat");
}

return 0;
}

```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment No. 3

Q.3) Write c program to find properties such as inode number,number of hard link,file permissions,file size,file access and modification time and so on of a given file using stat() function.

```
#include <stdio.h>
#include <sys/stat.h> // Required for stat() and struct stat
#include <time.h>     // Required for ctime()

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
        return 1;
    }

    struct stat fileStat;

    // Call stat() to get file information
    if (stat(argv[1], &fileStat) < 0) {
        perror("Error getting file stats");
        return 1;
    }

    printf("File: %s\n", argv[1]);
    printf("Inode Number: %ld\n", (long)fileStat.st_ino);
    printf("Number of Hard Links: %ld\n", (long)fileStat.st_nlink);
    printf("File Size: %lld bytes\n", (long long)fileStat.st_size);

    // File Permissions
    printf("File Permissions: ");
    printf((S_ISDIR(fileStat.st_mode)) ? "d" : "-");
    printf((fileStat.st_mode & S_IRUSR) ? "r" : "-");
    printf((fileStat.st_mode & S_IWUSR) ? "w" : "-");
    printf((fileStat.st_mode & S_IXUSR) ? "x" : "-");
    printf((fileStat.st_mode & S_IRGRP) ? "r" : "-");
    printf((fileStat.st_mode & S_IWGRP) ? "w" : "-");
    printf((fileStat.st_mode & S_IXGRP) ? "x" : "-");
    printf((fileStat.st_mode & S_IROTH) ? "r" : "-");
    printf((fileStat.st_mode & S_IWOTH) ? "w" : "-");
```

```

printf((fileStat.st_mode & S_IXOTH) ? "x" : "-");
printf("\n");

// Access and Modification Times
printf("Last Access Time: %s", ctime(&fileStat.st_atime));
printf("Last Modification Time: %s", ctime(&fileStat.st_mtime));
printf("Last Status Change Time: %s", ctime(&fileStat.st_ctime));

return 0;
}

```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment No .4

Q.4) Print the type and size of a given file.

```
#include<stdio.h>
#include<string.h>
#include<sys/stat.h>
void main()
{
    Struct stat filestat;
    FILE * f1;
    char s1[100];
    Printf("enter file name");
    Scanf("%s",&s1);
    f1=fopen(s1,"r");
    printf("size=%ld",filestat.st_size);
    printf("type=");
    if(strstr(s1,".html"))
        printf("html type");

    if(strstr(s1,".csv"))
        printf("csv type");

    if(strstr(s1,".php"))
        printf("php type");

    if(strstr(s1,".c"))
        printf("c type");
}
```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment No.5

Q.5) Write a c program to find wheather a given file is present in current directory or not.

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/stat.h>
#include<sys/types.h>
#include<dirent.h>
```

```
void main()
{
char fn [30];
```

```
int fd=0;
```

```
FILE *f1;
```

```
printf("enter path name::");
```

```
scanf("%s", &fn);
```

```
f1=fopen(fn,"r");
```

```

if(f1==NULL)
printf("not in current");
else
printf("in current");

}

```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment No.6

Q.6) Write a c program that accept a string as an argument and return all the files that begins with that name in the current directory .for e.g. ./a.out sss will return all file names that begins with sss.

```
#include<sys/stat.h>

#include<sys/types.h>

#include<dirent.h>

#include<string.h>
int main()

{

    DIR *dr;

    struct dirent *en;

    char s1[20];

    dr=opendir(".");

    printf("enter string to match");

    scanf("%s", &s1);

    if (dr)

    {

        while((en=readdir(dr))!=NULL)

        {
```

```

        if(strstr(en->d_name,s1)!=NULL)

        printf("\n %s", en->d_name);

    }

}

}

```

Assignment Evaluation

0: Not Done		1: Incomplet		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment No 7

Q.7) Read the current directory and display name of files ,no of files in current directory.

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<unistd.h>
```

```
#include<sys/stat.h>
```

```
#include<sys/types.h>
```

```
#include<dirent.h>
```

```
int main()
```

```
{
```

```
    DIR *dr;
```

```
    struct dirent *en;
```

```
    int c=0;
```

```
    dr=opendir(".");
```

```
    if(dr)
```

```
{
```

```
    while((en=readdir(dr))!=NULL)
```

```

{

printf("\n %s", en->d_name);
c++;
}

}
Printf("total no of files=%d",c);
}

```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment No.8

Q.8 Write c program to print all environment variables

```
#include<stdio.h>
int main(int argc, char *argv,char* envp)
{
    char ** env;
    for(env=envp;*env!=0 ;env++)

    {

    char * thisEnv=*env;

    printf("%s\n",thisEnv);

    }

    return (0);
}
```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment NO.9

Q.9) Atexit() system call execution for exit_handler().

```
#include<stdio.h>
```

```
void exit_handler1();
```

```
void exit_handler2();
```

```
int main()
```

```
{
```

```
int pid;
```

```
atexit(exit_handler1);
```

```
atexit(exit_handler2);
```

```
pid=fork();
```

```
if(pid=0)
```

```
exit(0);
```

```
else
```

```
{
```

```
sleep(2);
```

```
exit(0);
```

```
}
```

```
return 0;}
```

```
void exit_handler1()
```

```
{
```

```
printf("one exit\n");
```

```
}
```

```
void exit_handler2()
```

```
{
```

```
printf("two exit\n");
```

```
}
```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment NO.10

Q.10) Fork(),vfork() system call execution

```
#include<stdio.h>
int global;
int main()
{

    int local;
    local=1;
    if(vfork()==0)
    {

        global=2;

        local=3;

    }

    printf("global %d local %d\n",global,local);
    return(0);
}
```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment NO.11

Q.11) Write a C Program that demonstrates redirection of standard output to a file.

```
#include <stdio.h>

int main() {
    FILE *fp;

    // Redirect stdout to file "output.txt"
    fp = freopen("output.txt", "w", stdout);
    if (fp == NULL) {
        perror("freopen");
        return 1;
    }

    // This will go into output.txt instead of terminal
    printf("HELLO WORLD\n");
    printf("This is written to the file using redirection.\n");

    // Restore stdout to terminal
    freopen("/dev/tty", "w", stdout);
    printf("Back to terminal output.\n");

    return 0;
}
```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment NO.12

Q.12) Write a C program that illustrates suspending and resuming processes using signals.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        // ===== Child process =====
        while (1) {
            printf("Child process (PID=%d) is running...\n", getpid());
            sleep(1);
        }
    } else {
        // ===== Parent process =====
        sleep(3);
        printf("Parent: Suspending child (PID=%d)...\n", pid);
        kill(pid, SIGSTOP); // suspend child
        sleep(5);
        printf("Parent: Resuming child (PID=%d)...\n", pid);
        kill(pid, SIGCONT); // resume child

        sleep(3);
    }
}
```

```

    printf("Parent: Terminating child (PID=%d)...\n", pid);
    kill(pid, SIGKILL); // kill child
    wait(NULL);
    printf("Parent: Child terminated.\n");
}

return 0;
}

```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment NO.13

Q.13) Demonstrate the use of atexit() function.

```
#include <stdio.h>
#include <stdlib.h>

void goodbye1(void) {
    printf("Goodbye from function 1!\n");
}

void goodbye2(void) {
    printf("Goodbye from function 2!\n");
}

int main() {
    // Register functions with atexit()
    if (atexit(goodbye1) != 0) {
        fprintf(stderr, "Failed to register goodbye1\n");
        return 1;
    }
    if (atexit(goodbye2) != 0) {
        fprintf(stderr, "Failed to register goodbye2\n");
        return 1;
    }

    printf("Main function is running...\n");
    printf("Exiting program now...\n");

    return 0; // or exit(0);
}
```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment NO.14

Q.14) Message 1= "Hello SPPU".

```
#include <unistd.h>
int main()
{
    char msg[] = "Message 2 = Hello SPPU\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1); // system call
    return 0;
}
```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge

Assignment NO.15

Q.15) Message 3 = "Linux is Funny"

```
#include <unistd.h>
int main(void)
{
    const char msg[] = "Message 3 = Linux is Funny\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    return 0;
}
```

Assignment Evaluation

0: Not Done		1: Incomplete		2: Late Complete	
3: Need Improvement		4: Completed		5: Well Done	

Date:

Practical In-charge